

Exploring natural language processing in mechanical engineering education: Implications for academic integrity

International Journal of Mechanical
Engineering Education
2024, Vol. 52(1) 88–105
© The Author(s) 2023



Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/03064190231166665
journals.sagepub.com/home/ijj



Jonathan Lesage¹, Robert Brennan¹ ,
Sarah Elaine Eaton² , Beatriz Moya²,
Brenda McDermott³, Jason Wiens⁴,
and Kai Herrero¹

Abstract

In this paper, the authors review extant natural language processing models in the context of undergraduate mechanical engineering education. These models have advanced to a stage where it has become increasingly more difficult to discern computer vs. human-produced material, and as a result, have understandably raised questions about their impact on academic integrity. As part of our review, we perform two sets of tests with OpenAI's natural language processing model (1) using GPT-3 to generate text for a mechanical engineering laboratory report and (2) using Codex to generate code for an automation and control systems laboratory. Our results show that natural language processing is a potentially powerful assistive technology for engineering students. However, it is a technology that must be used with care, given its potential to enable cheating and plagiarism behaviours given how the technology challenges traditional assessment practices and traditional notions of authorship.

Keywords

Academic integrity, artificial intelligence, engineering, engineering education, academic misconduct, GPT-3, teaching, learning

¹Mechanical and Manufacturing Engineering, University of Calgary, Calgary, Canada

²Werklund School of Education, University of Calgary, Calgary, Canada

³Accessibility Testing, University of Calgary, Calgary, Canada

⁴English, University of Calgary, Calgary, Canada

Corresponding author:

Robert Brennan, Mechanical and Manufacturing Engineering, University of Calgary, Calgary, Canada.

Email: rbrennan@ucalgary.ca

Introduction

Natural language processing (NLP) is a field of artificial intelligence research and application that focuses on understanding and deciphering writings and generating human-like text.¹ In recent years there has been a leap in the capability of natural language processing models: particularly their ability to regularly generate text which humans struggle to identify as machine written.² Understandably, concerns have been raised around the societal impact of NLP, such as its role in generating fake news, spreading disinformation, or causing economic disruption.^{3,4} Concerns have also been raised around the impact these models may have on education and academic integrity. For example, the New York Times recently reported that NLP may pose challenges to educators attempting to prevent plagiarism and cheating.⁵ Such a possibility has direct implications on academic integrity and must be met directly by educators.

While policies on the use of artificial intelligence are likely best left to individual institutions, there is a clear issue with an NLP model being used to generate text which may be used as a response to an assignment at the instructor level. Given that NLP models are now generating code, and even solving word-based mathematics problems, these concerns extend to engineering curricula. Presently, there are no universally accepted definitions of plagiarism.⁶ Post-secondary institutions' definitions of plagiarism are typically explained in academic integrity policy documents, but there is little consistency across universities as to how they define plagiarism.⁷ However, many educators will likely consider it cheating for students to employ such a technology to generate responses and solutions on various assignments. More specifically, students using these tools would get credit for work they did not do.⁷ In such cases, it would be unlikely that traditional plagiarism checkers could be used to detect this form of cheating since the NLP model generates an AI-generated response that is unique (rather than copying an existing response). As such, an investigation of the technology, and its capability, is necessary to identify how it may affect engineering education and academic integrity.

In this paper, the authors explore the impact of NLP on teaching and learning in undergraduate mechanical engineering programs. We begin with an overview of the internal workings NLP technology, followed by an examination of current NLP models of interest. Next, we investigate the literature surrounding the practical tasks where the technology has been applied. This is followed by a set of experiments to assess how NLP models may be used by students in mechanical engineering laboratory and computing courses. We conclude with a discussion of the implications of NLP technologies on undergraduate mechanical engineering education.

An overview of natural language processing

NLP is a sub-field of linguistics, computer science, and artificial intelligence (AI) that focuses on several language tasks such as translation, sentence completion, sentiment identification, and text generation.⁸ The basic approach combines rule-based models of human language with machine learning and involves training a neural network on an extremely large data set (i.e., “deep learning”) with the goal of creating a model that understands both the context and intent of natural languages.⁸ Most current NLP

models are classified as “pre-trained transformer” models: i.e., “pre-trained” is in reference to the large data set that is used to train an NLP model, while “transformer” refers to the resulting pre-trained neural network’s ability to process an entire sequence of text at once (instead of one word at a time).^{1,8} For example, OpenAI’s latest Generative Pre-Trained Transformer model, GPT-3, was trained on text from five data sets (CommonCrawl, WebText2, Books1, Books2, and Wikipedia), representing more than five trillion words.² Given this massive training set, GPT-3 can generate reasonable solutions without any additional training by the user. This default model setting is referred to “zero-shot setting” or “zero-shot learning.” If a more focused solution is desired, the user can include examples of successful completions as part of their GPT-3 input: i.e., resulting in a “one-shot setting” (one example) or a “few-shot setting” (multiple examples). The user also has the option to create a “fine-tuned” model by pre-training GPT-3 with a set of examples.

Current models and their capabilities

To appreciate the potential impact of extant NLP models on engineering education, it is important to gain an understanding of their capabilities. This section provides a basic overview of these models. Wherever possible, we also provide reference to the industry-standard GLUE and SuperGLUE (General Language Understanding Evaluation) benchmark evaluation datasets as a basis of comparison.^{1,9}

First introduced in 2019, Google’s Bidirectional Encoder Representations from Transformers (BERT) model¹⁰ was intended to address the limitations of extant standard language models that were hindered by their unidirectional nature. GPT, for example, used a left to right architecture, meaning that each token (i.e., numerical representation of a word) could only attend to previous tokens in the layers of the transformer. In contrast, BERT alleviated that constraint by employing a masked language model which could consider tokens on the left and the right context. They believed this strategy would lead to a more efficient NLP model.

BERT established state-of-the-art performance on eleven NLP tasks, including outperforming GPT on the GLUE benchmark. Furthermore, it did this with a training set of 340 million parameters that were then fine-tuned for the specific benchmark task it was given. This was equivalent to GPT at the time and demonstrated that a bi-directional approach could be more efficient.

This approach was refined in 2020 with the introduction of the ELECTRA model¹¹ using a new strategy called replaced token detection. Like BERT, this is a masked language model that uses context from the left and right of a sentence. However, after pre-training, a second neural network, called a “discriminator,” is fine-tuned on the data. The authors¹¹ argued that this approach was more efficient because the pre-training task was defined over all input tokens rather than just those which were replaced.

Results from ELECTRA on GLUE suggest that the approach was another success in the NLP field. ELECTRA, while not achieving state-of-the-art performance, was able to outperform BERT and GPT on GLUE, while requiring significantly less training resources. As well, ELECTRA has been demonstrated to be an effective zero shot learner and could outperform BERT on many of these benchmark tasks.¹²

While the development of BERT and ELECTRA demonstrate that NLP models can increase in efficacy using innovative new pre-training methods and architectures, development on GPT-3 showed that increasing the size of pre-training data (i.e., to 175 billion parameters) could dramatically improve performance.² This was unprecedented in 2020, and GPT-3 demonstrated excellent performance on multiple data sets including SuperGLUE. In some cases, GPT-3 outperformed even fine-tuned models based on BERT. However, this required few-shot learning with the model. Depending on the task it was given, GPT-3 could achieve incredible performance with just a few in-context examples, demonstrating the performance benefits of a massive training set.

An early example of this success was a demonstration of GPT-3's ability to write short news articles that human evaluators could not accurately discern as AI-written.² This naturally concerned many, as it demonstrated that NLP models could easily generate human like text without fine-tuning, and it also made GPT-3 far more useful to individuals not specialised in NLP. Since then, many others have gone on to demonstrate the remarkable writing capability of GPT-3.³

While reasonable performance on typical language processing tasks was expected from GPT-3, researchers were surprised to see the model generated snippets of code.⁵ This resulted from the large training set which inadvertently included code. This led OpenAI to develop Codex,¹³ a GPT-3 model trained on Python code available from GitHub. Codex can now write basic Python code from natural language prompts, and the model can succeed at astonishing rates. Although the model cannot outperform humans on complex coding tasks, the researchers note that Codex "displayed strong performance on a dataset of human-written problems comparable to easy interview problems."¹³

In the spring of 2020, Google released the Pathways Language Model (PALM)¹⁴: another transformer model built using a training data set consisting of 540 billion parameters. With few-shot learning, this model achieved state-of-the-art performance on 28 of 29 benchmark tasks, further reinforcing the observation that large language models can achieve exceptional performance with few-shot learning because of their training data size. The performance increases from increased model size have yet to plateau, and so we may expect that if researchers increase the size of training data the performance of NLP models will continue to increase.¹⁴

While many more NLP models exist such as older variants of GPT-3 like the previously mentioned GPT and its successor GPT-2, or variants of BERT like RoBERTa, the models presented here demonstrate two important trends. First, innovative approaches in architecture and training data, like those applied in BERT and ELECTRA, may lead to more efficient models. Second, continuing to increase the training data set size will likely continue to produce more effective models. These trends likely mean that we will continue to see advancements in the field of NLP for years to come.

Practical applications of natural language processing

Although the models presented in the previous section are impressive, the various benchmark tests presented in the papers do little to demonstrate the practical application of the

models. In some cases, the results from the data sets are not applicable to real-life tasks of meaningful complexity. This issue was explored by van der Lee et al.⁹ when they sought to examine the best practices for humans evaluating machine-generated text. One example was the bilingual evaluation understudy (BLEU) metric, which is used to measure the quality of a translation between two languages by a machine. While a translation may score low on the BLEU metric, because it does not match a reference translation for a dataset, it may still be a valid translation. Similarly, the researchers' tasks of a benchmark test may not be as useful in real-world applications. Based on this, they recommended that more human evaluation of machine-generated text is necessary for evaluating the efficacy of models and listed several recommendations for conducting such evaluation.

Thus, in examining the technical capabilities of NLP models, it is necessary to examine some of the novel ways in which they are applied. This is not to state that benchmark tests are useless, as it is impractical to rely solely on human examination of NLP models and benchmark tests are an excellent way to reliably compare models with consistent metrics. To understand the implications that these models may have it is necessary to examine the manner in which they may be used to complete tasks more akin to those seen in a classroom.

Wu et al.¹⁵ at OpenAI developed an approach for generating summaries of novels using a fine-tuned model based on GPT-3. Their approach involved breaking down a novel into smaller sections and writing summaries of those smaller sections. This process continues until an overall summary is generated and then evaluated. The results were evaluated by humans, with 5% of summaries receiving a score of 6/7, and 15% receiving a score of 5/7 with the best produced model. Overall, many of the machine generated summaries were worse than those generated by humans, but the results are promising for NLP models completing complex tasks like this.

Crossley et al.¹⁶ applied NLP technology to evaluate summaries of various texts written by students. They applied four different openly available NLP models, and through their own modelling of the evaluation of results from these NLP models were able to categorise summaries as either high or low quality. Their approach successfully determined the quality of a summary 80% of the time and determined the level of education of the text at about the same frequency. While this contribution does not represent a new model, it does show an interesting way in which educators might apply NLP, and students may even vet copied text for quality.

The previous two applications required large amounts of human input, which is not always readily available. Imran et al.¹⁷ examined this challenge of sentiment analysis from student reviews of classroom teachings and professors. They developed an NLP model based on a smaller number of student reviews, which would then generate many student reviews, which were more balanced between positive and negative. Using this data, they generated another model focused on sentiment analysis, and showed that the approach improved accuracy. It is noteworthy that NLP generated prompts can be used to train more NLP models.

A unique use of NLP was displayed by Demirci et al.,¹⁸ who applied GPT-2 to detect static malware. Their approach involved fine-tuning GPT-2 and Stacked BiLSTM,

another pre-trained NLP model, to find malicious code. Their GPT-2 model performed exceptionally well, detecting malicious code 85.4% of the time. While this example may not apply directly to education outside of computer science, it does demonstrate the capability of these models once fine-tuning has been applied.

Perhaps the largest concern and obvious application of NLP models is the generation of human-like text. Salminen et al.¹⁹ examined this issue in regards to fake reviews of online products. Using GPT-2 as a base, they created a fine-tuned model to generate fake reviews of various products. Furthermore, they fine-tuned GPT-2 and RoBERTa to detect fake reviews. The results were particularly interesting when compared to human evaluations. Humans only detected fake reviews from the model around 55% of the time (i.e., a little better than chance). The GPT-2 and RoBERTa models, in contrast, detected fake reviews with 96% and 83% accuracy, respectively. Their work once again demonstrates the importance of fine-tuning models, but provides further evidence that AI generated text is able to evade detection by human readers.

The issue of fake reviews is also of concern for the tourism industry, and Tuomi²⁰ has conducted preliminary research into the issue, though his results are limited. The work he collected thus far once again suggests that humans struggle to detect AI-generated text.

The results from the work of Salminen et al.¹⁹ reinforces that of Brown et al.,² in that the short text generations from NLP models are difficult to detect by humans. However, Brown et al. note that the longer the text generations are more easily detected by humans. Thus, there remains a reasonable doubt of how effective NLP models would be at more academic tasks seen by students.

Kobis and Mossink²¹ provide some insight into this with their work on AI generated poetry. Their approach involved fine-tuning GPT-2 to write poetry. This model would be given two starting lines, and from there it would generate a full-length poem. With this set of poems, they completed two Turing tests. In the first, they had amateur poets write poems with the same two starting lines as the machine-generated poems, while in the second they used professionally written poems, like those of Maya Angelou, to compare with, again with the first two lines. The findings were of significant interest. In the first set of Turing tests, human judges detected the correct origin of the poem 50.21% of the time, making it effectively chance. When asked of their confidence in determining the origin of the poem, however, human judges had an average confidence level of 62.27 on a scale from 0 to 100. Humans were not only poor at detecting the origin of the poem, but as well were consistently confident in their selection. These results changed once the model was placed against professional poets, where human judges showed a preference towards the human written poems and detected the correct origin beyond chance levels.

Presently, the applications of NLP models have largely focused on detection or text generation of some form. However, there has been a desire to apply the technology towards mathematics. Cobbe et al.²² focused on this problem in 2021 at OpenAI. First, they collected and presented a dataset called GSM8 K, which consists of slightly over 8000 grade school mathematics problems. This is noteworthy, as it has become a frequently tested upon dataset. Until this point, NLP models had struggled with mathematics, as they were prone to small individual mistakes. The approach that Cobbe et al.²²

took was to apply verifiers. They used a fine-tuned model based on GPT-3 parameters to generate solutions and then had a verifier find the most frequent solution and elect this as the correct answer. The verifier approach (pre-trained with six billion parameters) performed as well as a fine-tuned model based on GPT-3 with 175 billion parameters, showing their approach was equivalent to increasing the pre-training data by a factor of 30. Unsurprisingly, their highest success rate, about 55%, was achieved by using fine-tuned generators and verifiers based on GPT-3's 175 billion parameter models.

Wei et al.²³ took a different approach to the challenge of solving mathematics problems, in particular word-based ones. Beginning with PaLM as their base model, they provided chain of thought prompting in the form of few and single-shot learning and achieved a remarkable success rate of 57% on GSM8 K. Their approach made use of the few shot learning capability that large language models exhibit, and their prompt simply walked through the problem solving process with the model the same way in which an educator would do so with a student. These results suggest that using prompts in this manner can elicit reasoning in a large language model.

Lewkowycz et al.²⁴ developed a model called Minerva, which was another fine-tuned model built from PaLM, trained on a high-quality data set consisting of numerous mathematics examples. Furthermore, Minerva used its own internal calculator. The fine-tuning process once again demonstrated exceptional performance, achieving a success rate of 78.5% on GSM8 K, establishing the current state of the art. Furthermore, it reinforced the importance of fine-tuning models to achieve exceptional, but select, performance.

As noted previously, the discovery that GPT-3 could generate simple programs from Python docstrings led to the development of a specialised GPT model, called Codex, that focuses on a variety of coding tasks.¹³ In their release paper on Codex, OpenAI note that Codex currently generates the 'right' code in 37% of use cases.¹³ Although Codex can generate correct code in many cases, clearly it requires close supervision on the part of the user. The success of the Codex project led to the development of CoPilot, a code completion module embedded in Microsoft Visual Studio.²⁵

A similar approach was followed by Google with PaLM with its "PaLM Coder" model.¹⁴ Given the larger size of PaLM's training set, it has been shown to outperform Codex on multiple coding datasets.

The applications of NLP are growing rapidly and span a wide range of application domains as well as a wide range of processing tasks. As well, these tools are becoming much more accessible. For example, in November 2021, OpenAI removed the wait list for its GPT-3 large language processing API, allowing the public to explore its capabilities and integrate the model into their app or service.²⁶ The API allows users to generate sequences of words starting from a source input called a prompt. The resulting completion is text that is statistically a good fit given the starting text (prompt). GPT-3's natural language processing capabilities are "shockingly good"²⁷ – as Floridi and Chiriatti²⁸ note, signal "the arrival of a new age in which we can now mass produce good and cheap semantic artifacts".

There are clearly two main avenues in which NLP can continue to improve in capability. The first of these is to continue applying innovative approaches to how transformer models work, as was done in the work on BERT and ELECTRA. The second, and more

straightforward manner, is to simply increase the size of pre-training data. This was clearly demonstrated in the work on PaLM, where the authors directly note that “scaling improvements from large [Language Models] have neither plateaued nor reached their saturation point.”¹⁴ All researchers must do to increase the efficacy of a model is to give it more pre-training data. Thus, we can, and should, expect that NLP technologies will continue to improve in capability with time.

It follows that it is only a matter of time before we see NLP tools of this type enter our students’ educational toolkit. In the next section, we provide two examples of how this could be accomplished with OpenAI’s GPT-3 and Codex models.

Two tests with OpenAI

In this section, we focus on two key writing activities performed by undergraduate mechanical engineering students: written material for laboratory reports, and industrial computer programming. The purpose of performing these tests is to explore the feasibility of a student completing an assignment using an NLP tool. If the task can be accomplished with little effort (i.e., no or little fine tuning) and with human-like output, then it may be conceivable for a student to use NLP tools in a similar way to complete an assignment. Thus, the tests will provide insight on whether NLP tools pose a possible threat to academic integrity in the undergraduate mechanical engineering education context.

For each test, the OpenAI API is provided with a short written prompt that is interpreted by the OpenAI execution engine to generate a completion. The execution engine determines the language model used by GPT-3 to perform the natural language processing task: we use text-davinci-003 for written material and code-davinci-001 for code generation. The GPT-3 response (i.e., completion) is affected by both its input (i.e., prompt) and the execution engine parameters. These parameters allow the user to control various factors such as the length of the response, the “creativity” of the response, and the consistency of responses (when multiple tests are performed).

Two key GPT-3 parameters are “temperature” and “Top P”. To understand how these two parameters work together it is important to note that NLP models are trained to predict the probability of the next word based on the input prompt.²⁹ If the model always samples the most likely next word, it runs the risk of getting stuck in loops. To get around this problem, sampling methods are based on sampling from a distribution; however, this also creates a problem, since sampling from a distribution can cause the model to go off topic (i.e., when it samples from the tail of the distribution).

When combined, the temperature and Top P variables allow the user to tune the model to generate sequences of arbitrary length (i.e., prevent it from getting stuck in loops) while staying on topic. Temperature, T , is inspired by statistical thermodynamics: at lower temperatures the model is increasingly confident of its top choices, while at higher temperatures the model moves towards uniform sampling. Mathematically, the sampling probabilities are obtained by dividing a logits function by T before feeding them into softmax function.³⁰ Top P, p , controls how much of the tail of the probability function is removed. Mathematically, the cumulative distribution is computed and cut off as soon as the cumulative distribution function (CDF) exceeds p .³⁰

From the user's perspective, these two parameters combine to influence the "creativity" of the response: i.e., T controls the randomness of the response, while p controls the scope of the randomness. For our experiments, we set Top P at its maximum value of 1 to allow GPT-3 to explore the entire spectrum of responses and control the randomness using the temperature dial.³¹

Writing

Mechanical engineering undergraduate students are engaged in a wide range of writing activities during their program. This example involves producing written text from laboratory notes taken in a senior fluids course. In this case, the student has made point-form notes during the laboratory session that include comments on the experimental results. The objective is to produce a written "Discussion" section based on the point-form notes.

To produce the text, we used OpenAI's "Playground" API with the GPT-3 text-davinci-003 engine ($T=0$, $p=1$). The prompt is shown in text at the top of Figure 1: it includes the phrase "Convert my notes into the 'Discussion' section for a laboratory report:" followed by a transcription of the student's point-form notes that were taken during the laboratory. The GPT-3 completion is shown highlighted in green at the bottom of Figure 1.

The prompt is a very straight forward example of zero-shot text generation. In this case, the student simply asks GPT-3 to convert their notes into a "Discussion" section (i.e., the first sentence), then includes the point-form notes from the laboratory. No training examples are included in the prompt.

Convert my notes into the "Discussion" section for a laboratory report: 

Object: to study the operating characteristics of a convergent-divergent nozzle

Apparatus: nozzle, static pressure probe, pressure transducer, pressure display, digital temperature sensor

Figure 1: expected external wave system observed using Schlieren apparatus

Figure 3: 3 different flow regimes observed (wholly subsonic, shockwave in divergent section, sonic at throat)

Figure 3: pressure distributions across nozzle agree with theoretical

Figure 3: limit values do not have p/p_0 ratio of 1.0; are lower instead (expected since pressure at nozzle entrance not equal to ideal case)

Figure 3: shockwave shows as "bump" in supersonic curves; unlike ideal curve, do not have infinite slopes at $M=1$; instead rapidly increasing at $M=1$

Results: calculated design exit Mach number are $M = 0.36$ (subsonic flow) and $M = 2.03$ (supersonic flow)

Results: for subsonic region, plenum pressure is 21 kPa, pitot pressure is 20.1 kPa; consistent

Results: for supersonic region, plenum pressure is 202 kPa, pitot pressure is 124.3 kPa; error likely caused by pitot tube measurement near shockwave (non-isentropic flow)

Discussion: The results of this experiment demonstrate the operating characteristics of a convergent-divergent nozzle. The expected external wave system observed using the Schlieren apparatus was consistent with the theoretical predictions (Figure 1). Three different flow regimes were observed (Figure 3): wholly subsonic, shockwave in the divergent section, and sonic at the throat. The pressure distributions across the nozzle agreed with the theoretical predictions, although the limit values did not have a p/p_0 ratio of 1.0; instead, they were lower than expected (Figure 3). This is likely due to the fact that the pressure at the nozzle entrance was not equal to the ideal case. Additionally, the shockwave showed as a "bump" in the supersonic curves, and unlike the ideal curve, did not have infinite slopes at $M=1$; instead, the slope rapidly increased at $M=1$. The calculated design exit Mach numbers were $M = 0.36$ (subsonic flow) and $M = 2.03$ (supersonic flow). For the subsonic region, the plenum pressure was 21 kPa and the pitot pressure was 20.1 kPa, which were consistent. For the supersonic region, the plenum pressure was 202 kPa and the pitot pressure was 124.3 kPa; this discrepancy is likely due to the pitot tube measurement near the shockwave, which is a non-isentropic flow.

Figure 1. The GPT-3 prompt and completion for the laboratory notes to "discussion" section example.

From a writing perspective, this completion can be assessed from the perspective of surface errors (i.e., grammar, spelling, syntax) and global errors (i.e., readability, comprehension).³² The resulting completion contains nine sentences with an average of 23 words per sentence that are free of surface errors. The resulting completion also scores well in terms of readability based on Flesch-Kincaid readability tests,³³ which calculate the ease with which a reader can comprehend a passage of text based on number of syllables per word and number of words per sentences. The text generated is as at a Flesch Reading Ease of 43 (i.e., “university” level of difficulty). One can also view the completion in terms of the genre of writing (i.e., writing an undergraduate engineering laboratory report). Based on the authors’ experience with undergraduate engineering laboratory reports, the GPT-3 completion would not look out of place in an undergraduate laboratory report and could be inserted directly into the laboratory report in its current form or an edited form. While this completion required the technical content to be prepared by the student (i.e., the laboratory notes included in the prompt), GPT-3 has saved the student the task of producing the expository text.

To determine the cognitive level of this learning activity, it is important to view the learning activity in the context of the intended learning outcome.³⁴ For example, if the intended learning outcome is to simply “summarise” the notes into text that is free of surface and global errors, this learning activity can be viewed at the “comprehension” level of Bloom’s taxonomy.³⁵ As noted previously, GPT-3 appears to be capable of this. However, it remains to be seen whether GPT-3 can produce writing that demonstrates higher-level functions such as analysis, synthesis, and evaluation. For example, if the intended learning outcome is to make conclusions about a lab experiment from the point form notes or combine the evidence together in a meaningful way, we cannot claim that GPT-3 has this capability. The implication here is that engineering students may be able to use NLP tools for simple tasks only, or as a “starting point” with significant editing for more complex writing.

```
PROGRAM thermistor
VAR
  Vout AT %IW100 : UINT;
END_VAR
VAR
  R1 : REAL := 10000.0;
  C1 : REAL := 1.009249522e-3;
  C2 : REAL := 2.378405444e-4;
  C3 : REAL := 2.0109202697e-7;
  Temp : REAL;
  LNR2 : REAL;
END_VAR
(* Calculate the thermistor resistance *)
LNR2 := LN(R1 * (65535.0 / (UINT_TO_REAL(Vout)) - 1.0));
(* Calculate the temperature using the Steinhart-Hart equation *)
Temp := 1.0 / (C1 + C2 * LNR2 + C3 * EXP(LNR2, 3)) - 273.0;
END_PROGRAM
```

Figure 2. The authors’ IEC 61131-3 structured text solution to the thermistor laboratory problem.

Programming

For the next set of tests, we focus on a laboratory exercise from a second-year undergraduate course in automation and controls taught by the authors. For this exercise, students were required to create a program to interface a thermistor to a programmable logic controller (PLC) using IEC 61131-3.³⁶ The exercise was conducted in a laboratory setting using the OpenPLC software platform³⁷ with the Arduino Uno R3 hardware platform.³⁸

Thermistors and resistance temperature devices (RTDs) are basically variable resistors that change resistance with temperature. Given that the Arduino Uno R3 utilises analogue voltage inputs, students needed to recognise that the varying resistance must first be converted to a voltage using a simple voltage divider circuit. The thermistor resistance could then be determined by solving the voltage divider transfer function,

$$R_{th} = R_1 \left(\frac{V_{in}}{V_{out}} - 1 \right) \quad (1)$$

where R_{th} is the thermistor resistance, R_1 is the series resistor (roughly equal to the maximum thermistor resistance), and V_{in} and V_{out} are the circuit's input and output voltages respectively.

```

1  (*
2  Create an IEC 61131-3 Structured Text program to measure temperature in Centigrade from a thermistor
3  *)
4
5  (*
6  The thermistor is connected to the PLC's analog input channel 0.
7  The thermistor is connected to a voltage divider circuit with a 10kOhm resistor.
8  The voltage divider circuit is connected to the PLC's 5V power supply.
9  *)
10
11  (*
12  The thermistor's resistance is measured using the voltage divider formula:
13
14  R_thermistor = R_resistor * (V_in / V_out - 1)
15
16  The thermistor's temperature is measured using the Steinhart-Hart equation:
17
18  1/T = A + B * ln(R_thermistor) + C * ln(R_thermistor)^3
19
20  The Steinhart-Hart coefficients are:
21
22  A = 0.001129148
23  B = 0.000234125
24  C = 8.76741E-08
25
26  The thermistor's temperature is measured in Centigrade.
27  *)
28
29  (*
30  The PLC's analog input channel 0 is configured as follows:
31

```

Figure 3. Codex “zero-shot” completion for the thermistor laboratory problem.

Once V_{out} is acquired and converted to the thermistor resistance using equation (1), code had to be written to convert the resistance to a temperature in degrees Celsius ($^{\circ}\text{C}$) using the Steinhart-Hart equation.³⁹ The authors' solution in IEC 61131-3 ST (structured text) notation is shown in Figure 2. For this implementation $V_{in}=5\text{ V}$ and $R_1=10\text{ k}\Omega$. OpenPLC uses a 16-bit convention for analog I/O.

For the Codex tests, we chose to focus on ST given that Codex generates text-based completions. The prompt and completion for the thermistor laboratory problem are shown in Figure 3.

The Codex prompt, shown at the top of Figure 3 (lines 1–3), is in the form of an ST comment (i.e., comments in ST are enclosed by “(“ and “)”). As can be seen from Figure 3, the completion (lines 5–30) was unsuccessful in this case. Rather than generating ST code, Codex generated additional comments. However, it is interesting to note that the comments show that Codex recognises that the voltage divider transfer function must be solved (lines 12–14) and that the Steinhart-Hart equation is required (lines 16–25) even though it cannot generate the required ST code. Given that Codex is trained on GitHub, it is likely that there are insufficient IEC 61131-3 examples to provide successful completions in this set of languages. However, when prompted to generate an Arduino

```

1  /* Create an Arduino program to measure temperature in Centigrade from a thermistor */
2
3  #include <Arduino.h>
4
5  #define THERMISTORPIN A0
6  #define THERMISTORNOMINAL 10000
7  #define TEMPERATURENOMINAL 25
8  #define BCOEFFICIENT 3950
9  #define SERIESRESISTOR 10000
10
11 void setup() {
12   Serial.begin(9600);
13 }
14
15 void loop() {
16   int a = analogRead(THERMISTORPIN);
17   float resistance = (float)(1023-a)*SERIESRESISTOR/a;
18   float steinhart;
19   steinhart = resistance / THERMISTORNOMINAL; // (R/Ro)
20   steinhart = log(steinhart); // ln(R/Ro)
21   steinhart /= BCOEFFICIENT; // 1/B * ln(R/Ro)
22   steinhart += 1.0 / (TEMPERATURENOMINAL + 273.15); // + (1/To)
23   steinhart = 1.0 / steinhart; // Invert
24   steinhart -= 273.15; // convert to C
25   Serial.print("Temperature = ");
26   Serial.print(steinhart);
27   Serial.println(" *C");
28   delay(1000);
29 }

```

Figure 4. The Codex Arduino solution to the thermistor laboratory problem. The Codex prompt is shown on line 1. The Codex completion in C is shown on lines 3 to 29.

program, Codex generates a successful completion in C as shown in Figure 4. With this solution, Codex recognises both the need to solve the voltage divider transfer function and the need to use the Steinhart-Hart equation.

Comparing the Codex implementation to the authors' implementation, both successfully convert the thermistor input to a temperature reading in Celsius. The main difference between the solutions is in the representation of the analog input, V_{out} , to the Arduino: i.e., the authors' use a 16-bit word while Codex uses a 10-bit word. Given that the Codex prompt specified an Arduino program, the 10-bit word is correct since it is consistent with the Arduino's 10-bit analog-to-digital converter. Our solution uses a 16-bit word for consistency with the OpenPLC analog I/O convention. The two implementations also differ in the choice of task interval; however, this is an implementation detail that could be updated at runtime.

At this stage, the student could choose to translate the C program (Arduino solution) to IEC 61131-3 ST. Arguably, this would be a straight-forward task given the close correspondence between each language's syntax. However, this may not always be the case:

```

1  (*
2  Write a program in IEC 61131-3 Structured Text to calculate the average and standard
3  deviation of three temperature inputs.
4  *)
5  PROGRAM TempSensors
6  VAR
7      Temp1 AT %IW100 : UINT;
8      Temp2 AT %IW101 : UINT;
9      Temp3 AT %IW102 : UINT;
10 END_VAR
11 VAR
12     T1 : REAL;
13     T2 : REAL;
14     T3 : REAL;
15     ave_temp : REAL;
16     stdev_temp : REAL;
17 END_VAR
18 (*
19     Scale the temperature inputs:
20     Temp1, Temp2, Temp3: 0 to 65535
21     T1, T2, T3: -20 to 200 C
22 *)
23 T1 := (220.0 / 65535.0) * UINT_TO_REAL(Temp1) - 20.0;
24 T2 := (220.0 / 65535.0) * UINT_TO_REAL(Temp2) - 20.0;
25 T3 := (220.0 / 65535.0) * UINT_TO_REAL(Temp3) - 20.0;
26 (* Calculate average *)
27 ave_temp := (T1 + T2 + T3) / 3.0;
28 (* Calculate standard deviation *)
29 stdev_temp := SQRT((EXPT((T1 - ave_temp),2) + EXPT((T2 - ave_temp),2) + EXPT((T3 - ave_temp),2)) / 3.0);
30 END_PROGRAM
31 (*
32 Write a program in IEC 61131-3 structured text to read a temperature
33 in Celcius from a thermistor.
34 *)

```

Figure 5. The prompt for a one-shot setting Codex solution to the thermistor laboratory problem using a structured text “statistics” example. This prompt includes the example's prompt (lines 1–4), the example completion (lines 5–30), and the prompt for the thermistor program (lines 31–34).

```

35 PROGRAM Thermistor
36   VAR
37     thermistor AT %IW100 : UINT;
38   END_VAR
39   VAR
40     R : REAL;
41     T : REAL;
42   END_VAR
43   (*
44     Scale the thermistor input:
45     thermistor: 0 to 65535
46     R: 0 to 100000 Ohms
47   *)
48   R := (100000.0 / 65535.0) * UINT_TO_REAL(thermistor);
49   (*
50     Use the Steinhart-Hart equation to calculate the temperature from the resistance:
51     http://en.wikipedia.org/wiki/Steinhart%E2%80%93Hart_equation
52   *)
53   T := (1.0 / ((1.0 / 298.15) + (1.0 / 3435.0) * LN(R / 10000.0)
54     + (1.0 / 676125.0) * LN(R / 10000.0) * LN(R / 10000.0) * LN(R / 10000.0))) - 273.15;
55 END_PROGRAM

```

Figure 6. The completion of a one-shot setting Codex solution to the thermistor laboratory problem using a structured text “statistics” example (lines 35–55).

especially when the solution requires an understanding of the real-time behaviour of the automation system.

Given that GPT-3 can be trained for specific use cases,⁴⁰ it should be possible to generate an ST completion directly. The trade-off, however, will be between training effort and translation effort. A very large training set (i.e., “fine-tuning”) would result in a model that would perform as well with ST as it does with Python and C. However, it is highly unlikely that this approach would be used by students to solve the thermistor laboratory problem. A more reasonable approach would be to provide GPT-3 with a small number of ST examples (i.e., one-shot or few-shot setting) that it could use to understand the syntax for the specific use case.

To test this approach, we began with a simple “statistics” example that includes key elements of the IEC 61131-3 ST syntax for this problem. More specifically, the prompt shown in Figure 5 includes information on the specification of variables, the scaling of inputs, type conversions, and mathematical operations.

The resulting successful completion is shown in Figure 6. This example illustrates the power of the GPT-3 Codex model: with only a single example, it can build on its built-in training set to infer a solution for this very specialised use case. This does not mean that a solution to any IEC 61131-3 problem can be achieved with a one-shot setting; however, by carefully selecting related examples, a solution can be obtained.

Conclusions

Given the rapid advances in NLP models, it is likely – if not inevitable – that engineering educators will encounter this technology in the classroom. Regardless of whether this will take the form of an assistive technology (e.g., to support diverse learners with access

concerns) or a disruptive technology (e.g., to enable cheating behaviours), it will need to be addressed directly by faculty and their institutions. Our investigations into the current state of NLP have shown that extant models are capable of producing results for general natural language (i.e., free of surface and global errors) and coding tasks (i.e., at the level of easy interview problems¹³) but require additional context-specific training to be applied to typical undergraduate mechanical engineering problems. For example, OpenAI is quite capable of converting laboratory notes to written text but requires additional training to generate code for an industrial automation system.

From the student perspective, NLP technology is currently very accessible, both as a stand-alone API (e.g., the OpenAI Playground⁴¹ and ChatGPT⁴²) and as a plug-in to existing engineering tools (e.g., the VisualStudio CoPilot plug-in²⁵). For learners who struggle with surface errors, such as those with learning disabilities or second language learners, this tool might reduce barriers that impact their ability to express their technical knowledge. However, the main barrier lies in the effort required to provide these models with the necessary context-specific training to make them applicable to undergraduate mechanical engineering problems. Based on our review of the literature, there are two paths to achieve this. The first, fine-tuning, can clearly achieve excellent results as Kobis and Mossink demonstrate with their AI generated poetry using a fine-tuned model based on GPT-2.²¹ The obvious barrier for students is that a proper fine-tuned model requires thousands of examples, and thus, is labour intensive. In this way, NLP tools need fine-tuning to adapt the specific contexts and genre requirements of discipline-specific tasks.

We believe it is unlikely that students would employ such a strategy on their own. The amount of work required to create a fine-tuned model is well beyond that of a typical assignment or paper. Where it may become of some concern is in contract cheating, a scenario in which students pay for someone else to complete their assigned task and submit it as their own.⁴³ Should an individual or organisation create a fine-tuned model capable of completing select assignments, students may readily use it to complete their work. Given the high efficacy, but highly specialised nature, of these models they would likely only find use in niche situations, thus necessitating the development of many different fine-tuned models to cover a wider range of tasks.

The second way NLP responses may be improved to a more useful form, one- or few-shot setting, was demonstrated in the previous section. The results from GPT-3² and from PaLM¹⁵ support the use of few-shot learning to increase the performance of models on datasets. Literature surrounding the use of this method for practical applications, however, is limited at best.

Conceivably, students could use an NLP platform (e.g., OpenAI's GPT-3) and quickly generate a useful response to a question with few-shot learning. They would require similar questions and responses, which could be gathered from former students, textbooks, or online sources, and be given as input to a model. The model could then conceivably generate a useful response. Such a methodology could be easily deployed by students as demonstrated by the previous industrial automation system coding example using Codex. In their present state, NLP models do not appear to pose a significant threat to academic integrity in the engineering classroom given the extra work required

to move beyond zero-shot learning. Since NLP platforms rely on the existing training and data, they do have the potential to challenge instructors in their assessment designs. Repeated use of the same assignments and structures increase the likelihood of the NLP platforms creating reasonable responses. Unfortunately, it is not possible to “design out” cheating by changing assessments; however, Bretag et al.⁴⁴ showed that some assessment types are less susceptible to academic misconduct. Specifically, authentic assessments such as those requiring demos, oral exams, or practical examinations may be less susceptible to breaches of academic integrity, but there is no such thing as a cheat-proof assessment.⁴⁴ For engineering educators, the key will be understanding the capabilities of NLP as this technology advances, and applying this knowledge to our design and application of intended learning outcomes, learning activities, and assessments.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This research study is funded by the University of Calgary / Office of the Vice-Provost (Teaching and Learning) and by the National Sciences and Engineering Research Council of Canada (NSERC) through grant CDE486462-15.

ORCID iDs

Robert Brennan  <https://orcid.org/0000-0003-0468-393X>

Sarah Elaine Eaton  <https://orcid.org/0000-0003-0607-6287>

References

1. Lauriola I, Lavelli A and Aiolfi F. An introduction to deep learning in natural language processing: Models, techniques, and tools. *Neurocomputing* 2022; 470: 443–456.
2. Brown TB, Mann B, Ryder N, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 2020; 2020-Decem, 2005.14165.
3. Tamkin A, Brundage M, Clark J, et al. Understanding the capabilities, limitations, and societal impact of large language models. *arXiv* 2021: 1–8. URL <http://arxiv.org/abs/2102.02503>. 2102.02503.
4. Christian B. *The alignment problem: Machine learning and human values*. New York, New York: Norton, 2020.
5. Johnson SAI. Is mastering language. should we trust what it says? *The New York Times* 2022; 1–21.
6. Eaton SE. Plagiarism in higher education: tackling tough topics in academic integrity. *Libraries Unlimited*, an imprint of ABC-CLIO, LLC, 2021.
7. Eaton SE. Comparative analysis of institutional policy definitions of plagiarism: A pan-Canadian university study. *Interchange: A Quarterly Review of Education* 48: 271–281. <https://doi.org/10.1007/s10780-017-9300-7>

8. Kublick S and Saboo S. *GPT-3: Building innovative NLP products using large language models*. 1st ed. Sebastopol, California: O'Reilly, 2022.
9. van der Lee C, Gatt A, van Miltenburg E, et al. Human evaluation of automatically generated text: Current trends and best practice guidelines. *Comput Speech Lang* 2021; 67. DOI:10.1016/j.csl.2020.101151
10. Devlin J, Chang MW, Lee K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding. NAACL HLT 2019–2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference 2019; 1(Mlm): 4171–4186. 1810.04805.
11. Clark K, Luong MT, Le QV, et al. ELECTRA: Pre-training text encoders as discriminators rather than generators. *arXiv* 2020; 1–18. URL <http://arxiv.org/abs/2003.10555>. 2003.10555.
12. Ni S and Kao HY. ELECTRA is a zero-shot learner, too. *arXiv* 2022; URL <http://arxiv.org/abs/2207.08141>. 2207.08141.
13. Chen M, Tworek J, Jun H, et al. Evaluating large language models trained on code. *ArXiv* 2021. URL <http://arxiv.org/abs/2107.03374>. 2107.03374.
14. Chowdhery A, Narang S, Devlin J, et al. PaLM: Scaling language modeling with pathways. *arXiv* 2022; 1–83 URL <http://arxiv.org/abs/2204.02311>. 2204.02311.
15. Wu J, Ouyang L, Ziegler DM, et al. Recursively summarizing books with human feedback. *arXiv* 2021; URL <http://arxiv.org/abs/2109.10862>. 2109.10862.
16. Crossley SA, Kim M, Allen L, et al. Automated summarization evaluation (ASE) using natural language processing tools. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 2019; 11625 LNAI: 84–95.
17. Imran AS, Yang R, Kastrati Z, et al. The impact of synthetic text generation for sentiment analysis using GAN based models. *Egypt Inform J* 2022. DOI: 10.1016/j.eij.2022.05.006
18. Demirci D, Sahin N, Sirlancis M, et al. Static malware detection using stacked BiLSTM and GPT-2. *IEEE Access* 2022; 10: 58488–58502.
19. Salminen J, Kandpal C, Kamel AM, et al. Creating and detecting fake reviews of online products. *Journal of Retailing and Consumer Services* 2022; 64: 102771.
20. Tuomi A. Deepfake consumer reviews in tourism: Preliminary findings. *Annals of Tourism Research Empirical Insights* 2021; 2: 100027. URL
21. Kobis N and Mossink LD. Artificial intelligence versus Maya Angelou: Experimental evidence that people cannot differentiate AI-generated from human-written poetry. *Comput Human Behav* 2021; 114. DOI:10.1016/j.chb.2020.106553
22. Cobbe K, Kosaraju V, Bavarian M, et al. Training verifiers to solve math word problems. *arXiv* 2021; 1–22 URL <http://arxiv.org/abs/2110.14168>. 2110.14168.
23. Wei J, Wang X, Schuurmans D, et al. Chain of thought prompting elicits reasoning in large language models. *arXiv* 2022; 1–41 URL <http://arxiv.org/abs/2201.11903>. 2201.11903.
24. Lewkowycz A, Andreassen A, Dohan D, et al. Solving quantitative reasoning problems with language models. *arXiv*; 1–54arXiv:2206.14858v2.
25. VisualStudio. CoPilot for VS Code, 2023. URL <https://marketplace.visualstudio.com>.
26. Wiggers T. OpenAI makes GPT-3 generally available through API. *Venture Beat* 2021; URL <https://venturbeat.com/2021/11/18/>.
27. Heaven W. Openai's new language model gpt-3 is shockingly good - and completely mindless. *MIT Technology Review* 2020; URL <https://www.technologyreview.com>.
28. Floridi L and Chiriatti M. Gpt-3: its nature, scope, limits, and consequences. *Minds and Machines* 2020; 3: 681–694.
29. Shree P. The journey of OpenAI GPT models. *Medium* 2020 URL <https://medium.com/walmartglobaltech/the-journey-of-open-ai-gpt-models-32d95b7b7fb2>.

30. Holtzman A, Buys J, Du L, et al. The curious case of neural text degeneration. *ArXiv* 2019; URL <https://doi.org/10.48550/arxiv.1904.09751>.
31. Abd-Elaal E-S, et al. Assisting academics to identify computer generated writing. *European Journal of Engineering Education* 2022; 47: 725–745.
32. Montgomery J and Baker W. Teacher-written feedback: Student perceptions, teacher self-assessment, and actual teacher performance. *Journal of Second Language Writing* 2007; 16: 82–99.
33. Flesch R. A new readability yardstick. *Journal of Applied Psychology* 1948; 32: 221–233.
34. Biggs J and Tang C. *Teaching for quality learning at university*. New York, New York: McGraw Hill, 2011.
35. Bloom's Taxonomy of Measurable Verbs. *UTICA University*, 2023. URL <https://www.utica.edu/academic/Assessment/new/Blooms%20Taxonomy%20-%20Best.pdf>
36. Commission IE. IEC 61131-3 programmable controllers - part 3: programming languages, 2013. URL <https://webstore.iec.ch/publication/4552>.
37. OpenPLC. Open source PLC software, 2022. URL <https://openplcproject.com>.
38. Arduino. Arduino Uno Rev3, 2022. URL <https://store-usa.arduino.cc>.
39. Steinhart J and Hart S. Calibration curves for thermistors. *Deep-Sea Res Ocean Abst* 1968; 15: 497–503.
40. Solaiman I and Dennison C. Process for adapting language models to society (palms) with values-targeted datasets. Creative Commons 2021.
41. OpenAI. GPT-3 Application Programming Interface, 2023. URL <https://openai.com/api/>.
42. OpenAI. Introducing ChatGPT, 2023. URL <https://openai.com/blog/chatgpt>.
43. Lancaster T. Commercial contract cheating provision through micro-outsourcing web sites. *International Journal for Education Integrity* 2020; 16: 1–14.
44. Bretag T, Harper R, Burton M, et al. Contract cheating and assessment design: Exploring the relationship. *Assessment & Evaluation in Higher Education* 2019; 44: 676–691.